Bafna Gold

Program 1

1. Write a program in C to find second smallest element in an array.

Program :-

```
#include <stdio.h>
#include <limits.h>

int main() {
    int n, i;
    printf("Enter the number of elements in array");
    scanf("%d", &n);
    if (n<2) {
        printf("Array must have atleast two elements\n");
        return 1;
    }
    int arr[n];
    printf("Enter %d elements", n);
    for (i=0; i<n; i++) {
```

```
        scanf("%d", &arr[i]);
    }
    int smallest = INT_MAX;
    int second.Smallest = INT_MAX;
    for (i=0; i<n; i++) {
        if (arr[i] < smallest) {
            scond smallest = Smallest;
            smallest = arr[i];
        }
        else if (arr[i] < secondSmall
                && arr[i] != smallest) {
            secondSmallest = arr[i];
        }
    }
    if (secondSmall == INT_MAX) {
        printf("There is no 2nd
            smallest elem (all elem are
            equal)");
    } else {
        printf("The 2nd Small is
```

```
/ d n", secondSmallest);
}
return 0;
}
```

Output:-

```
Enter number of element in array: 3
Enter 3 element:
3
6
8
The second smallest element is: 6
```

Program 2:

2. Write a program in C to find the sum of the left diagonal of a matrix

Program

```
#include <stdio.h>
int main () {
    int n, i, j, sum=0;
    printf("Enter size of square matrix");
    scanf("%d", &n);
    int matrix[n][n];
    printf("Enter %d x %d matrix elements\n", n, n);
    for (i=0; i<n, i++) {
        for (j=0; j<n; j++) {
            scanf("%d", &matrix[i][
        }
    }
```

```
for (i=0; i<n; i++) {
    sum += matrix[i][i];
}
printf("Sum of left diagonal
    (prim diaga) elem %d\n",
        :sum);
return 0;
}
```

Output:-

```
Enter size of a squai matrix : 2
Enter 2x2 matrix elem :
3
6
9
1
```

Sum of left diagonal (primay diagonal) element : 4

Program 3:

3. Write a program in C to find the sum of row a column of a matrix

Program :-

```
#include <stdio.h>
int main() {
    int rows, cols, i, j;
    printf("Enter number of rows");
    scanf("%d", &rows);
    printf("Enter number of columns");
    scanf("%d", &cols);
    int matrix[rows][cols];
    printf("Enter %dx%d matrix
        element \n", row, col);
    for (i=0; i<rows; i++) {
        for (j=0; j<cols; j++) {
            scanf("%d", &matrix[i][j
```

```
    }}
    printf("\n Sum of each row \n");
    for (i=0; i<rows; i++) {
        int rowSum = 0;
        for (j=0; j<cols; j++) {
            rowSum += matrix[i][j];
        }
        printf("Row %d: %d\n", i+1,
                rowSum);
    }
    printf("\n Sum of each Coleumn: \n");
    for (j=0; j<cols; j++) {
        int ColSum=0;
        for (i=0; i<rows; i++) {
            colSum += matrix[i][j];
        }
        printf("Coloumn %d: %d\n", j+1,
                colSum);
    }
    return 0;
}
```

Program 4

4. Write a program in C to count the total number of duplicate elements in an array.

Program :-

```
#include <stdio.h>
int main() {
    int n, i, j, count = 0;
    printf("Enter number of \n", n);
    for (i = 0 i < n; i++) {
        Scanf("%d", &arr[i]);
    }
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (arr[i] == arr[j]) {
                count++;
            }
        }
    }
}
```

~~return 0~~

printf("Total num of duplicate elements %d\n", count);

return 0;

}

Program 15

5. Write a program in C to find second largest element in an array

Program :-

```
#include <stdio.h>
#include <limits.h>

int main () {
    int n, i;

    printf ("Enter number of elements in the array");
    scanf ("%d", &n);
    if (n<2) {

        printf ("Array must have atleast 2 element \n");
        return 1;
    }

    int arr[n];

Printf ("Enter %d element \n", n);
Scanf ("%d", &arr[i]);
} for (i=0; i<n; i++) {
        Scanf ("%d", &arr[i]);
    }
```

```
int largest = INT_MAX
int secondlargest = INT_MIN;

for (i=0; i<n; i++) {
    if (arr[i] > largest) {
        secondlargest = largest;
        largest = arr[i];
    }
    else if (arr[i] > secondlargest &&
             arr[i] != largest) {
        secondlargest = arr[i];
    }

if (secondlargest == INT_MIN) {
    printf ("The is no second
    largest element (all elements
    are equal \n");
} else {
    printf ("The second largest element
    is %d \n", secondlargest);
}

return 0;
}
```

Program 6

6. Write a program in C to delete an element at a desired post from an array.

Program:

```
#include <stdio.h>
int main()
{
    int n, i, position;

    printf("Enter the number of
        elements in an array:");
    scanf("%d", &n);
    int arr[n];

    printf("Enter %d elements: \n", n);
    for(i=0; i<n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter the position to delete
            (1 to %d): ", n);
    scanf("%d", &position);
```

```
if (positio < 1 || position > n) {
    printf("Invalid positio ! Posid
    must be between 1 and
    %d \n", n);
    return 1;
}

position = position - 1;

for (i = position; i < n - 1; i++) {
    arr[i] = arr[i+1];
}

printf("Array after delete element at
position %d \n", position+1);
for (i = 0; i < n - 1; i++) {
    printf("%d", arr[i]);
}
printf("\n");
return 0;
}
```

Week 2

Q Write a C program to simulate cpu scheduling algorithm

1) FCFS
2) SJF

Program:-

```
#include <stdio.h>
#define MAX 20
void fcfs (int n, int at[], int bt[]){
    int wt[MAX], tat[MAX], ct[MAX],
        rt[MAX];
    int time = 0;
    float total_wt = 0, total_tat = 0,
    total_rt = 0;
    for (int i = 0; i < n; i++) {
        if (time < at[i])
            time = at[i];
        rt[i] = time - at[i];
        wt[i] = rt[i];
        time += bt[i];
        ct[i] = time;
        tat[i] = ct[i] - at[i];
```

```
    total_wt += wt[i];
    total_tat += tat[i];
    total_rt += rt[i];
}

printf("\n--- FCFS ---\n");
printf("PID\tAT\tBT\tCT\tWT\
    \tTAT\tRT\n");

for (int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\
        %d\t%d\t%d\n",
    i+1, at[i], bt[i], ct[i], wt[i]
    tat[i], rt[i]);
}

printf("\nAverage Waiting Time = %.2f",
    total_wt/n);

printf("\nAverage Turnaround Time = %.2f",
    total_tat/n);
printf("\nAverage Response Time = %.2f\n",
    total_rt/n);

void sjf_nonpreemptive(int n, int at[],
    int bt[]) {

    int wt[MAX] = {0}, tat[MAX] =
        {0}, ct[MAX] = {0},
    rt[MAX] = {0};
```

```
int Complete [MAX] = {0};
int time = 0, count = 0;
float total_wt = 0, total_bt = 0, total_tt = 0

while (count < n) {

    int min = 9999, idx = -1;

    for (int i = 0; i < n; i++) {
        if (at [i] <= time && !Complete [i]) {
            if (bt [i] < min) {
                min = bt [i];
                idx = i;
            }
        }
    }

    if (idx != -1) {
        tt [idx] = time - at [idx];
        wt [idx] = tt [idx];

        time += bt [idx];
        ct [idx] = time;
        tat [idx] = ct [idx] - at [idx]

        total_wt += wt [idx];
        total_tat += tat [idx];
        total_tt += tt [idx];
        Complete [idx] = 1;
        count ++;
    }
```

```
    else {
        time ++;
    }
}

printf ("\n ---- SJF Non-Preemptive ----\n");
printf (" PID \tAT \tBT \tCT \tWT \tTAT \tRT \n");

for (int i = 0; i<n; i++) {
    printf (" %d \t %d \t %d \t %d \t %d \t %d \t %d \n",
        i+1, at[i], bt[i], ct[i], wt[i],
        tat[i], rt[i]);
}

printf ("\n Average Waiting Time = %.2f",
        total_wt/n);
printf ("\n Average Turnaround Time = %.2f",
        total_tat/n);
printf ("\n Average Respn Time = %.2f \n",
        total_rt/n);

void sjf_preemptive (int n, int at[],
                     int bt[]) {
    int rt=rem[MAX], wt[MAX]
        = {0}, tat[MAX] = {0};
    int first_start[MAX];
```

```
for (int i = 0; i < n ; i++) {
    rt = rem[i] = bt[i];
    first_start[i] = -1;
}

int complete = 0, time = 0;
float total_wt = 0, total_tat = 0,
total_rt = 0;

while (complete < n) {

    int min = 9999, idx = -1;
    for (int i = 0; i < n; i++) {
        if (at[i] <= time && rem[i] > 0) {
            if (rem[i] < min) {
                min = rem[i];
                idx = i;
            }
        }
    }
    if (idx == -1) {
        time++;
        continue;
    }

    if (first_start[idx] == -1)
        first_start[idx] = time;
    rem[idx]--;
    time++;
    if (rem[idx] == 0) {
        complete++;
        ct[idx] = time;
```

```
tat[idx] = ct[idx] - at[idx];
wt[idx] = tat[idx] - bt[idx];
rt[idx] = first_start[idx] - at[idx];

total_wt += wt[idx];
total_tat += tat[idx];
total_rt += rt[idx];
}

printf("\n--- SJF Preemptive (SRTF) ---\n");
printf("PID\tAT\tBT\tCT\tWT\tTAT\tRT\n");

for(int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i],
        wt[i], tat[i], rt[i]);
}

printf("\nAvg Waiting Time = %.2f",
    total_wt/n);

printf("\nAvg Turnaround Time = %.2f",
    total_tat/n);

printf("\nAvg Response Time = %.2f\n",
    total_rt/n);
```

```
int main () {

    int n, choice;
    int at[MAX], bt[MAX];

    printf ("Enter number of process: ");
    scanf ("%d", &n);

    for (int i = 0; i<n; i++) {
        printf ("\n Process %d \n", i+1);
        printf (" Arrival Time: ");
        scanf ("%d", &at[i]);
        printf ("Burst Time : ");
        scanf ("%d", &bt[i]);
    }

    printf ("\n Choose Algorithm ");
    printf ("\n1. FCFS");
    printf ("\n2. SJF Non-Preemptive");
    printf ("\n3. SJF Preemptive (SRTF)");
    printf ("\n Enter Choice");
    scanf ("%d", &choice);

    switch (choice) {

        case 1 :
            fcfs (n, at, bt);
            break;

        case 2 :
            sjf_nonpreempt (n, at, bt);
            break;
```

```
case 3:
    sjf_preemph(n, at, bt);
    break;

default:
    printf("Invalid choice");
}
return 0;
}
```

Output :-

Enter number of process : 4

Process : 1
Arrival Time : 0
Burst Time : 6

Process 2
Arrival Time : 1
Burst Time : 4

Process 3
Arrival Time : 2
Burst Time : 2

Process 4

Arrival Time : 3
Burst Time : 1

Choose Algorithm

1. FCFS
2. SJF Non-Preemptive
3. SJF Preemptive (SRTF)

Enter Choice : 1

--- FCFS ---

PID	AT	BT	CT	WT	TAT
1	0	6	6	0	6
2	1	4	10	5	9
3	2	2	12	8	10
4	3	1	13	9	10

Average Waiting Time = 5.50
Average Turnaround Time = 8.75
Average Response Time = 5.50

--- SJF Non-Preemptive ---

PID	AT	BT	CT	WT	TAT	RT
1	0	6	6	0	6	0
2	1	4	13	8	12	8
3	2	2	9	5	7	5
4	3	1	7	3	4	3

Average Waiting Time = 4.00
Average Turnaround Time = 7.25
Average Response Time = 4.00

--- SJF Preemptive (SRTF) ---

PID	AT	BT	CT	WT	TAT	RT
1	0	6	13	7	13	0
2	1	4	8	3	7	3
3	2	2	4	0	2	0
4	3	1	5	1	2	1

Average Wait Time = 2.75
Average Turnaround Time = 6.0
Average Response Time = 0.25

Comparison :-

SJF preemptive is the best, the SJF non preemptive then FCFS is the slowest

Algorithm	Avg WT	Avg TAT	Avg RT
FCFS	5.25	10.25	5.25
SJF (non preem)	4.50	9.50	4.50
SJF (preem)	4.25	9.25	3.25

Week 3

Priority and RoundRobin :

Code :-

```
#include <stdio.h>
#define MAX 20

void priority_np(int n, int at[], int bt[], int pr[])
{
    int wt[MAX] = {0},
        tat[MAX],
        rt[MAX],
        done[MAX] = {0};

    int time = 0, count = 0;

    while (count < n) {
        int idx = -1, min = 9999;
        for (int i = 0; i < n; i++)
            if (at[i] <= time &&
                !done[i] && pr[i] < min)
            { min = pr[i]; idx = i; }

        if (idx != -1) {
            rt[idx] = time - at[idx];
```

```
time t = bt[idx];
tat[idx] = wt[idx] + bt[idx];
done[idx] = 1; count++;
}
else time++;
}

printf("\n PID WT TAT RT \n");
int sum_wt = 0, sum_tat = 0, sum_rt = 0;
for (int i = 0; i < n; i++) {
    printf("%d %d %d %d \n", i+1,
           wt[i], tat[i], rt[i]);
    sum_wt += wt[i]; sum_tat += tat[i];
    sum_rt += rt[i];
}

printf("Average WT = %.2f \n", (float)
       sum_wt/n);

printf("Avg TAT = %.2f \n", (float) sum_tat/n);

printf("Average RT = %.2f \n", (float) sum_rt/n);
}

void priority_p(int n, int at[], int bt[],
                int pr[]) {
    int rt = bt[MAX], wt[MAX] = {0},
    tat[MAX], rt[MAX], flag[MAX] = {0};

    for (int i = 0; i < n; i++) { rt_bt[i] =
        bt[i]; rt[i] = -1;
```

```
int time = 0, complete = 0;
while (complete < n) {
    int idx = -1, min = 9999;

    for (int i=0; i<n; i++)
        if (at[i] <= time && rt[i] > 0 && pr[i] < min) {
            min = pr[i]; idx = i;
        }

    if (idx != -1) {
        if (rt[idx] == -1) rt[idx] =
            time - at[idx];

        rt_bt[idx] -= 1;
        time++;

        if (rt_bt[idx] == 0) {
            complete++;
            tat[idx] = time - at[idx];
            wt[idx] = tat[idx] - bt[idx];
        }
    }
    else time++;
}
```

```
printf("\n PID WT TAT RT \n");
int sum_wt=0, sum_tat=0, sum_rt=0;
for (int i=0; i<n; i++) {
    printf("%d %d %d %d \n", i+1, wt[i],
           tat[i], rt[i]);
    sum_wt += wt[i]; sum_tat += tat[i];
    sum_rt += rt[i];
}

printf("Average WT = %.2f \n", (float)sum_wt/n);
printf("Average TAT = %.2f \n", (float)sum_tat/n);
printf("Avg RT = %.2f \n\n", (float)sum_rt/n);

printf("\n Gantt Chart : \n");
for (int i=0; i<g_len; i++)
    printf("| P%d |", gantt[i]);
printf("\n 0");
for (int i=0; i<g_len; i++)
    printf("  P%d |", g_tim[i]);
printf("\n");
}

int main() {
    int n, choice, q;
    int at[MAX], bt[MAX], pr[MAX];
    printf("Processes ");
    scanf("%d", &n);

    for (int i=0; i<n; i++) {
        printf("AT BT PR for P%d : ", i+1);
        scanf("%d %d %d", &at[i], &bt[i],
              &pr[i]);
```

```
printf("\n 1. Priority Non-Preempt");
printf("\n 2. Priority Preemptive");
printf("\n 3. Round Robin");
printf("\n Enter Choice:");
scanf("%d", &choice);

switch(choice){
    Case 1: priority_np(n,at,bt,pr); break;
    Case 2: priority_p(n,at,bt,pr); break;
    case 2: priority_p("preempt");
    case 3: printf("Quanta:");
            scanf("%d",&q);
            rr(n,bt,q); break;
}
return 0;
}
```

Output

Process : 4

AT BT for P1 : 0 5 2

AT BT PR for P2 : 1 3 1

AT BT PR for P3 : 2 4 3

AT BT PR for P4 : 3 2 2

1. Priority Non-Preemptive
2. Priority Preemptive
3. Round Round

Choice : 1

Gant Chart

P1	P2	P4	P3

PID	WT	TAT	RT
1	0	5	0
2	4	7	4
3	8	12	8
4	5	7	5

Avg WT = 4.25
Avg TAT = 7.75
Avg RT = 4.25

Choice : 2

Gantt Chart :

P1	P2	P2	P4	P1	P4	P1	P1	P4	P4	P3	P3	P3	P3

PID	WT	TAT	RT
1	3	8	0
2	0	3	0
3	8	12	8
4	5	7	5

Avg WT = 4.00
Avg TAT = 7.50
Avg RT = 3.25

Choice 3
Quantum : 2

Gantt Chart

P1	P2	P3	P4	P1	P2	P3	P1

PID	WT	TAT	RT
1	9	14	0
2	8	11	2
3	9	13	4
4	6	8	6

Avg WT = 8.00
Avg TAT = 11.50
Avg RT = 3.00

Week 4
Multilevel Queue

Code:-

```
#include <stdio.h>
#define MAX 50

struct process {
    int pid;
    int at, bt, rt;
    int ct, wt, tat;
    int type;
};

struct queue {
    int items[MAX];
    int front, rear;
};

void initQueue(struct Queue *q) {
    q->front = q->rear = -1;
}

int isEmpty(struct Queue *q) {
    return (q->front == -1);
}

void enqueue(struct Queue *q, int val) {
    if (q->rear == MAX-1)
        return;
    if (q->front == -1) q->front = 0;
```

```
q->items[++q->rear] = val;
}

int dequeue(struct Queue *q) {
    if (isEmpty(q)) return -1;
    int val = q->items[q->front++];
    if (q->front > q->rear)
        q->front = q->rear = -1;
    return val;
}

void sortbyArrival(struct Process p[],
                   int n) {

    struct Process temp;
    for (int i=0; i<n-1; i++) {
        for (int j=i+1; j<n; j++) {
            if (p[i].at > p[j].at) {

                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
}}}}
```

```
int main() {
    int n;
    struct Process p[MAX];

    printf("Enter no. of process: ");
    scanf("%d", &n);

    for (int i=0; i<n; i++) {
        printf("\n Process %d \n", i+1);
        printf("PID: ");
        scanf("%d", &p[i].pid);
        printf("Arrival time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time");
        scanf("%d", &p[i].type);
        p[i].rt = p[i].bt;
    }

    sortbyArrival(p,n);
    struct Queue systQ, userQ;
    init Queue(&systemQ);
    init Queue(&userQ);

int time=0, complete=0, i=0;
int current = -1;

int gantt_pid[200], gantt_time[200];
int g=0;

while (completed<n) {
    while (i<n && p[i].at<=time) {
```

```
if (p[i].type == 1)
    enqueue (&sysQ, i);
else
    enqueue (&usrQ, i);
i++;
}
if (current != -1) {
  if (!isEmpty(&sysQ))
    current = dequeue(&sysQ);
  else if (!isEmpty(&usrQ))
    current = dequeue(&usrQ);
  else {
    time++;
    continue;
  }}
gantt_pid[g] = p[current].pid;
gantt_time[g] = time;
g++;
p[current].rt--;
time++;
if (p[current].rt == 0) {
    p[current].ct = time;
    completed++;
```

current = -1;
}

float total_wt = 0, total_tat = 0;
for (int i = 0; i < n; i++) {
p[i].tat = p[i].ct - p[i].at;
p[i].wt = p[i].tat - p[i].bt;
total_wt += p[i].wt;
total_tat += p[i].tat;
}

printf("\n Gantt Chart : \n");
for (int i = 0; i < g; i++) {
printf("P%d |", gantt_pid[i]);
}

printf("\n 0");
for (int i = 0; i < g; i++)
printf("%d", gantt_tim[i+1]);
}

printf("\n\n PID \t AT \t BT \t CT \t TAT
\t WT \n");

for (int i = 0; i < n; i++)
printf("P%d \t %d \t %d \t %d \t %d \t %d
\t %d \n",
p[i].pid, p[i].at, p[i].bt,
p[i].ct, p[i].tat, p[i].wt);
}

```
printf("\nAvg WT = %.2f",
        total_wt/n);

printf("\nAvg TAT = %.2f\n", total
        tat/n);

return 0;
}
```

Output 1

Enter Numb of process : 4

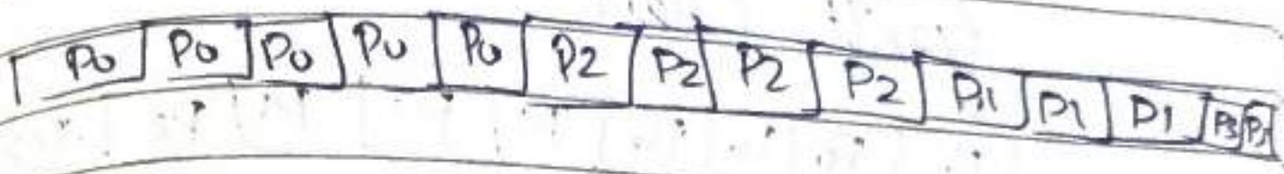
Process 1
PID = 0
ArrivalTim = 0
BurstTim = 5
Type (1 = Syst, 0 = Usr) : 1

Process 2
PID = 1
Arrival Tim : 2
Burst Tim : 3
Type (1 : Syst, 0 = Usr) : 0

Process 3
PID : 2
Arrival Tim : 1
Burst Tim : 4
Type (1 = Syst, 0 = Usr) : 1

Process 4
PID: 3
Arrival Tm : 3
Burst Tm : 2
Typ (1 = Sys, 0 = Usr) : 0

Gantt Chart



PID	AT	BT	CT	TAT	WT
P0	0	5	5	5	0
P1	1	4	9	8	4
P2	2	3	12	10	7
P3	3	2	14	11	9

Avrg WT = 5.00
Avrg TAT = 8.50

Output 2

Enter numbr of prosses : 3
Procss 1
PID: 0
AT : 0
Burst Tim: 4
Typ (1 = Sys, 0 = Usr): 1

Proses 2
PID: 1
AT : 1
Burst Tm : 3

Type (1 = Syst, 0 = Usr) : 1

Proc 3
PID = 2
Arrival Time : 2
Type (1 = System, 0 = User) : 1

Gantt Chart

PID	AT	BT	CT	TAT	WT
P0	0	4	4	4	0
P1	1	3	7	6	3
P2	2	2	9	7	5

Average WT = 2.67
Average TAT = 5.67

Output 3

Week 5

Code :-

```
#include <stdio.h>
#include <
#define MAX 10

typedef struct {
    int id;
    int bt;
    int deadline;
    int period;
    int ct, wt, tat;
    int done;
} Process;

void printGantt(int order[], int time_mark[],
                int size) {
    printf("\nGantt Chart\n");
    for (int i=0; i<size; i++)
        printf("| P%d ", order[i]);
    printf("|\n");
    printf("0 ");
    for (int i = 0; i<size; i++)
        printf("  %d ", time_mark[i]);
    printf("\n");
}
```

```
void edf(Proces p[], int n) {
    printf("\n --- Earlie Deadlin
                first (EDF) Schedul

    int time = 0, completed = 0;
    int order[MAX], time_mark[MAX],
        k = 0;

    for (int i = 0; i < n; i++)
        p[i].done = 0;

    while (complete < n) {
        int idx = -1, min_deadl = 9999,

    fo
```

```
float calculate_utilizatio (Proces P[], int n) {
    float U = 0;
    for (int i = 0; i < n; i++) {
        U += (float) p[i].bt/p[i].pe
    }
    return U;
}
```

```
void check_schedulablity (Proces p[], int n) {
    float U = calculd_utilizat (p, n);
    float bound = n*(pow(2, 1.0/n) - 1);

    printf("\n ---- Schedulabl
                test ---- \n");

    printf("\n Total Utilizat (U) =
                %.3f \n", U);
```

```
printf("\n[RMS]\n");
printf("Bound = %.3f\n", bound);

if (U <= bound)
    printf("Schedulable under RMS\n");
else
    printf("Not guaranteed Schedulable
        under RMS\n");

printf("\n[EDF]\n");
if (U <= 1.0)
    printf("Schedulable under
        EDF\n");
else
    printf("Not schedulable
        under EDF\n");

void edf(Process p[], int n) {
    printf("\n--- Earliest Deadline first
        (EDF) scheduling ---\n");
    int time = 0, completed = 0;
    int order[MAX], time_mark[MAX],
        k=0;
    for (int i=0; i<n; i++) p[i].done=0;
    while (completed < n) {
        int idx = -1, min_deadline =
                                    9999;
```

```
for (int i=0; i<n; i++) {
    if (!p[i].done && p[i].
        deadline < min_dead)
    {
        min_dead = p[i].deadline;
        idx = i;
    }
}

time += p[idx].bt;
order[k] = p[idx].id;
time_mark[k] = time;
k++;

p[idx].ct = time;
p[idx].tat = p[idx].ct;
p[idx].wt = p[idx].tat - p[idx].bt;

p[idx].done = 1;
complete++;
}

print_gantt(order, time_mark, k);
printf("\nID \tDeadline \tCT.
    \t WT \t TAT \n");

for (int i=0; i<n; i++) {
    printf("%d \t %d \t \t %d \t %d
        \n",
    p[i].id, p[i].deadline,
    p[i].ct, p[i].tat, p[i].wt
}
```

```
void rms( Process p[], int n) {
printf("\n---- Rate Monotonic
        Scheduling (RMS) ----\n");
int time = 0, completed = 0;
int order[MAX], time_marks[MAX],
        k = 0;

for(int i=0; i<n; i++) p[i].done
                = 0;

while (completed < n) {

int idx = -1, min_period
                = 9999;

for (int i = 0; i<n; i++) {
    if (!p[i].done && p[i].
        period < min_period) {

        min_period = p[i].period;
        idx = i;
    }
}
time += p[idx].bt;
order[k] = p[idx].id;
time_mark[k] = time;
k++;

p[idx].ct = time;
p[idx].tat = p[idx].ct;
p[idx].wt = p[idx].tat -
        p[idx].bt;
```

```
p[idx].done = 1;
completed++;
}

print_gantt(orders, time_max, b);

printf("\n ID\t BT\t Per\t CT\t WT
\t TAT\n");

for (int i = 0; i < n; i++) {

printf("%d\t %d\t %d\t %d\t %d\t %d\t
\n",

p[i].id, p[i].bt, p[i].
~~ct~~ p[i].tat ~~cat~~
p[i].period, p[i].ct, p[i].wt,
p[i].tat;

}

int main() {
int n;
Proces p[MAX], temp[MAX];
printf("Enter number of process:");
scanf("%d", &n);

printf("\n Enter proces Detail \n");
for (int i = 0; i < n; i++) {
p[i].id = 1 + i;
```

```
printf("Burst Tm :");
scanf("%d", &p[i].deaden);
printf("Period (for RMS) : ");
scanf("%d", &p[i].peri);
}

check = schedulability(p,n);
for(int i=0; i<n; i++) temp[i]=p[i];
edf(temp,n)

for(int i=0; i<n; i++)
    temp[i]=p[i];
rms(temp,n);
return 0;
}
```

Output

```
Enter Numb of process: 3
Enter Process Details:
Process 0:
Burst Tm : 1
Deaden (for EDF) : 4
Period (for RMS) : 4
```

Process 1:
Burst Time : 2
Deadline (for EDF) : 5
Period (for RMS) : 5

Process 2:
Burst Time : 1
Deadline (for EDF) : 7
Period (for RMS) : 7

--- Schedulably Test ---

Total Utilized (U) = 0.793

[RMS]
Bound = 0.730
Not guaranteed under RMS

[EDF]
Scheduled under EDF

Week 6 - Producer Consumer & Dining Philosopher

Code :-

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int in=0, out=0
sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int item;
    for (int i=0; i<10; i++) {
        item = rand() % 100;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced : %d at %d\n",
               item, in);
        in = (in+1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}
```

Week 6

```
void* Consumer (void* arg) {
    int item;
    for (int i=0; i<10; i++) {
        sem_wait (&full);
        pthread_mutex_lock (&mutex);
        item = buffer [out];
        printf ("Consumed: %d from %d
                \n", item, out);
        out = (out+1) % BUFFER_SIZE;
        pthread_mutex_unlock (&mutex);
        sem_post (&empty);
        sleep (2);
    }
    return NULL;
}

void* Consumer (void* arg) {
    int item;
    for (int i=0; i<10; i++) {
        sem_wait (&full);
        pthread_mutex_lock (&mutex);
        item = buffer [out];
        printf ("Consumed: %d from
                buffer [%d] \n", item,
                out);
        out = (out+1) % BUFFER_SIZE;
        pthread_mutex_unlock (&mutex);
        sem_post (&empty);
        sleep (1);
    }
```

```
return NULL;
}

void run_producer_consumer(){
    pthread_t p, c;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_creat(&c, NULL, &consum, NULL
    pthread_join(p, NULL);
    pthread_join(c, NULL);

#define N 5
sem_t fork[N];
void* philosopher(void* num){
    int id = *(int*)num;
    int left = id;
    int right = (id+1)%N;
    for(int i=0; i<3; i++){
        printf("Philosopher %d is thinking
                \n", id);
        sleep(1);
        if(id == N-1){
            sem_wait(&fork[right]);
            printf("Philosopher %d picked
                up right fork %d\n",
                id, right);

            sem_wait(&fork[left]);
            printf("Philosopher %d picked up
                left fork %d\n", id, left);
```

```
} else {
    sem_wait(&fork[left]);
    printf("Philosopher %d picked up
        left fork %d \n", id, left);

    sem_wait(&fork[right]);
    printf("Philosopher %d picked up right
        fork %d \n", id, right);
}

printf("Philosopher %d is eating \n",
        id);

sleep(2);
sem_post(&fork[left]);
sem_post(&fork[right]);
printf("philosopher %d put down fork
        %d and %d. \n", id, left,
            right);
}
return NULL;
}

void run_dining_philosph() {
    pthread_t ph[N];
    int id[N];
    for(int i=0; i<N; i++)
        sem_init(&fork[i], 0, 1);
    for(int i = 0; i<N; i++) {
        id[i] = i;
        pthread_create(&ph[i], NULL,
            philosph, &id[i]);
    }
```

```
for (int i=0; i<N; i++)
    pthread_join (ph[i], NULL);
}

int main ()
{
    int choice;
    printf ("\n -- Synchronized Simulation --\n");
    printf ("1. Produ-Consu Probl \n");
    printf ("2. Dining Philosophers Problem\n");
    printf ("Enter your choice:");
    scanf ("%d", &choice);
    if (choice == 1)
        run_produr_consumer ();
    else if (choice == 2)
        run_dining_philosopher ();
    else
        printf ("Invalid Choice \n");
    return 0;
}
```

Outputs:

1. Produr Consum
2. Dining Philosophers

Enter your choice: 1

Produced 0 at buffer [0]
Consumed 0 at buffer [0]

Produced : 1 at buff [1]
Consum : 1 from buff [1]

Produced : 2 at buff [2]
Consumed : 2 from buffer [2]

Produced : 3 at buffer [3]
Consumed : 3 from buffer [3]

Produced : 4 at buffer [4]
Consumed : 4 at from buffer [4]

Produced : 5 at buffer [0]
Consumed : 5 from buffer [0]

Produced : 6 at buffer [1]
Consum : 6 from buffer [1]

Produced : 7 at buffer [2]
Consumed : 7 from buff [2]

Produced : 8 at buffer [3]
Consumed : 8 from buffer [3]

Produced : 9 at buffer [4]
Consumed : 9 from buffer [4]

Produced : 10 : at buffer [1]
Consumed : 10 from buffer [1]

Produced : 11 at buffer [2]
Consu : 11 from buffer [2]

Enter your choice : 2
Philosopher 0 is thinking
Philosopher 4 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 picked up left fork 3
Philosopher 3 picked up left fork 4
Philosopher 2 picked up left fork 2
Philosopher 0 picked up left fork 0
Philosopher 3 is eating
Philosopher 1 picked up left fork 1
Philosopher 3 put down fork 3 and 4
Philosopher 3 is thinking

Week :- 7

Write a C program to simulate
a) Banker's algorithm for the purpose of deadlock avoidance

Code :-

```
#include <stdio.h>
#define MAX 10

int n, m;
int alloc [MAX][MAX], max[MAX][MAX],
    need [MAX][MAX];

int avail [MAX]
int isSafe() {
    int work[MAX], finish [MAX] = {0},
        safeSeq [MAX];
    int i, j, count = 0;

    for (i=0; i<m; i++)
        work[i] = avail [i];

    while (count < n) {
        int found = 0;

        for (i=0; i<n; i++) {
            if (!finish [i]) {
                int flag = 0;
                for (j=0; j<m; j++) {
                    if (need [i][j] > work
                        [j]) {
```

```
            flag = 1;
            break;
        }
    }
    if (!flag) {
        for (y=0; y<m; y++)
            work[y] += alloc[i][y];

        SafeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}

if (!found)
    return 0;

printf("\n Safe Sequence:");
for (i=0; i<n; i++)
    printf("%d ", SafeSeq[i]);
printf("\n");
return 1;
}

int main() {
    int i, j, pid;
    int Request[MAX];

    printf("Enter number of processes:");
    scanf("%d", &n);
```

```
printf("Enter number of resource: ");
scanf("%d", &m);

printf("\n Enter allocation matrix");
for (i=0; i<n; i++)
    for (j=0; j<m; j++)
        scanf("%d", &max[i][j]);

printf("\n Enter available resources:\n");
for (i=0; i<m; i++)
    scanf("%d", &avail[i]);

for (i=0; j<m; j++)
    need[i][j] = max[i][j] -
                 alloc[i][j];

printf("\n NEED Matrix:\n");
for (i=0; i<n; i++) {
    for (j=0; j<m; j++)
        printf("%d", need[i][j]);
    printf("\n");
}

if (isSafe())
    printf("\n System is initially in
        SAFE State\n");

else {
    printf("\n System is not safe\n");
    return 0;
}
```

```
printf("\n Enter process number making
        request ");
scanf("%d", &pid);

printf("Enter request vector \n");
for(i=0; i<m; i++)
   scanf("%d", &request[i]);

for(i=0; i<m; i++) {
   if(request[i] > need[pid][i]) {
      printf("\n Error: Request exceeds
              maximum need \n");
      return 0;
   }
}

for(i=0; i<m; i++) {
   avail[i] -= request[i];
   alloc[pid][i] += request[i];
}

if(isSafe())
   printf("\n Request can be granted
           (SAFE) \n");
else {
   printf("\n Request cannot be
           granted (UNSAFE) \n");

   for(i=0; i<m; i++) {
      avai[i] += request[i];
      alloc[pid][i] -= request[i];
      need[pid][i] += request[i];
   }
   return 0;
}
```

Output :-

I/P

Processes : 5

Resources : 3

Allocation :

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Max :

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Available

3 3 2

Request :

Process 4

Request 1 0 2

O/P

NEED MATRIX :

7 4 3

1 2 2

6 0 0

0 1 1

4 3 1

Safe Sequence : P1 P3 P4 P0 P2

System is initially in Safe State

Enter process making request : 1

Enter request vector :

1 0 2

After Request Processing

Safe Sequence : P1 P3 P4 P0 P2

Request can be granted (SAFE)

Unsafe

Available

1 0 0

System not Safe

Request Cannot be granted

Week - 8

Write a C program to simulate deadlock detection

Code:-

```
#include <stdio.h>
#define MAX 10

int main() {
    int n, m;
    int alloc[MAX][MAX], request[MAX]
                                [MAX];
    int avail[MAX], work[MAX], finish[MAX]
    int i, j;
    printf("Enter number of process and resources");
    scanf("%d %d", &n, &m);
    printf("Enter Allocation Matrix");
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Request Matrix");
    for (i=0; i<m; i++) {
        scanf("%d", &avail[i]);
        work[i]= avail[i];
    }
    printf("Enter Available Resources:\n");
    for(i=0; i<n; i++) {
        int allZero = 1;
        for (j=0; j<m; j++) {
            if (alloc[i][j] != 0) {
```

```
            allZero = 0;
            break;
        }
    }

    finish[i] = allZero ? 1 : 0;
}

int changed = 1;
while (changed) {
    changed = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            int canRun = 1;
            for (j = 0; j<m; j++) {
                if (request[i][j] > work[j]) {
                    canRun = 0;
                    break;
                }
            }

            if (canRun) {
                for (j=0; j<m; j++)
                    work[j] += alloc[i][j];
                finish[i] = 1;
                changed = 1;
            }
        }
    }
}

int deadlock = 0;
for (i=0; i<n; i++) {
    if (finish[i] == 0) {
        printf("P %d", i);
```

```
            deadlock = 1;
        }
    }
    if (!deadlock)
        printf("No Deadlock");
    return 0;
}
```

Output :-

Enter number of processes and resources : 3 2

Enter allocation matrix :

```
1  0
0  1
1  1
```

Enter Request matrix :

```
0  1
1  0
1  0
```

Enter Available Resources

```
0  0
```

Result

P0 P1 P2

No Deadlock

Enter number of process and resources: 3 3

Enter allocation Matrix:

0 1 0
2 0 0
3 0 3

Enter Request Matrix:

0 0 0
2 0 2
0 0 0

Enter Available Resources:

0 0 0

Result:

No Deadlock

Week 9

Write a C program to simulate the following Contiguous memory allocation techniques

a) Worst-fit
b) Best-fit
c) First-fit

Code:-

```
#include <stdio.h>
void firstFit(int blockSize[], int blocks,
    int processSize[], int processes)
{
    int allocation[processes];

    for(int i=0; i<processes; i++)
        allocation[i] = -1;

    for(int i=0; i<processes; i++)

    for(int j=0; j<blocks; j++)
    {
        if(blockSize[j] >=
            processSize[i])
        break;
    }
}
```

```
printf("\n Fist fit Allocation \n");
printf("Processes No \t Proces Size \t
        Block No \n");

for (int i=0; i<Processes; i++)
{

    printf("\n First fit Allocation \n");
    printf("Proces %d -> Block %d \n",
                                    j++;
        a[i]++);

    else
        printf("Proc %d -> Not
                allocated \n", i+

}

Void worst fit (int b[], int m,
        int p[], int n.) {

    int a[n];
    for(int i=0; i<n; i++)
      a[i]=-1;
    for (int i=0; i<n; i++)
        int worst = -1;
      for (int j=0; j<m;
                j++).

        if (b[j]>0 block size[
                worst[dx]
      worstIdx = j;
```

```
    }
  }
  if (worstIdx == -1 || blockSize[j] >
          blockSize[worstIdx])
      worstIdx = j;
  }
  if (worstIdx != -1) {
    allocation[i] = worstIdx;
    blockSize[worstIdx] = 0;
  }
}
printf("\n--- Worst Fit ---\n");
printf("ProcessNo.\tProcess Size\tBlock
        No.\n");
for (int i=0; i<n; i++) {
  printf("%d\t\t%d\t\t", i+1,
          processSize[i]);
  if (allocation[i] != -1)
    printf("%d\n", allocation[i]+1);
  else
    printf("Not allocated\n");
}
}

int main() {
  int n, m;
  printf("Enter number of memory
          blocks:");
  scanf("%d", &m);
  int blockSize[m];
```

```
printf("Enter number of process: ");
scanf("%d", &n);
int processSize[n];
printf("Enter Sizes of %d processes\n", n);
for(int i = 0; i < n; i++)
    scanf("%d", &processSize[i]);
int blockCopy1[m], blockCopy2[m],
    blockCopy3[m];
for(int i = 0; i < m; i++) {
    blockCopy1[i] = blockSize[i];
    blockCopy2[i] = blockSize[i];
    blockCopy3[i] = blockSize[i];
}

firstFit(blockCopy1, m, processSize, n);
bestFit(blockCopy2, m, processSize, n);
worstFit(blockCopy3, m, processSize, n);
return 0;
}
```

Output:

Enter Sizes of memory blocks:

100 500 200 300 600

Enter number of processes: 4

Enter size of processes:

212 417 112 426

First Fit Allocation :

Process No	Process Size	Block No
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

Best fit Allocation :

Process No	Process Size	Block No
1	212	4
2	417	2
3	112	3
4	426	5

Worst fit Allocation :

Process No	Process Size	Block No
1	212	5
2	417	2
3	112	5
4	426	Not Allocated

```
for (int i = 0; i < capacity; i++)
    frames[i] = -1;
printf("\n FIFO page replacement \n");
for (int i=0; i<n; i++)
{
    if (!isPresent(frames, capacity,
    page[i]))
    {
        frame[index] = pages[i];
        index = (index+1) % capacity;
        faults++;
    }
    printf("Page %d → ", page[i]);
    for (int j=0; j<capacity;
         j++)
    {
        if (frames[j] != -1)
            printf("%d", frame[j]);
        else
            printf("-");
    }
    printf("\n");
}
printf("Total Page faults = %d \n",
       faults);
}
```

Week 10

Write a C program to simulate page replacement algorithms

a) FIFO
b) LRU
c) Optimal

Code:-

```
#include <stdio.h>
#define MAX 50
int isPresent(int frame[], int n,
              int page)
{
    for (int i=0; i<n; i++)
    {
        if (frame[i]==page)
            return 1;
    }
    return 0;
}
void FIFO(int page[], int
          n, int capacity)
{
    int frame[MAX], index=0;
    faults=0;
```

```
void LRU(int pages[], int n,
         int capacity)
{
    int frames[MAX], time[MAX];
    int faults = 0, counter = 0;

    for (int i = 0; i < capacity; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\n LRU Page Replacement\n");

    for (int i = 0; i < n; i++)
    {
        int found = 0;

        for (int j = 0; j < capacity; j++)
        {
            if (frames[j] == pages[i])
            {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }

        if (!found)
        {
            int pos = 0;
```

```
for {int j = 1 ; j < capacity ; j++)
    if time[j] < time[pos])
        pos = j ;
    }

    counter ++ ;
    frame [pos] = page [i];
    time [pos] = counter ;
    faultH++ ;
}

printf ("Page %d →", page[i]);
for (int j=0; j<capacity ; j++)
    {
    if (fram [j] != -1)
        printf ("%d", frame[j]);
    else
        printf(" - ");
    {
    printf ("\n");
}

printf ("Total Page Faults = %d \n",
    faults);
}

Void Optimal (int page [], int n,
    { int capacity)

    int frame [MAX];
    int fault = 0;
```

```
for (int i=0; i<capacity; i++)
    frames[i] = -1;

printf("\n Optimal page
        replacement \n");

for (int i = 0; i<n; i++)
{
    if (!isPresent(frames, capacity,
        pages[i]))
    {

        int pos = -1, farthest = i+1;
        for (int j = 0; j<capacity; j++)
        {
            int k;
            for (k=i+1; k<n;
                k++)
            {
                if (frames[j] ==
                pages[k])
                {

                    if (k>farthest)
                    {
                        farthest = k;
                        pos = j;
                    }
                    break;
                }
            }
```

```
if (pos == -1)
    pos = 0;

frame[pos] = pages[i];
faults++;
}
printf("Page %d -> ", pages[i]);
for (int j=0; j<capacity; j++)
{
    if (frame[j] != -1)
        printf("%d", frame[j]);
    else
    { printf("-");
    }
printf("\n");
}
printf("Total page faults =
        %d\n", faults);
}

int main() {
    int pages[MAX], n, capacity;
    printf("Enter number of
        pages");
    scanf("%d", &n);
    printf("Enter page reference
        string\n");
```

```
for (int i = 0; i < n; i++)
    scanf("%d", &pages[i]);

printf("Enter frame capacity: ");
scanf("%d", &capacity);

FIFO(pages, n, capacity);
LRU(pages, n, capacity);
Optimal(pages, n, capacity);

return 0;
}
```

Output

FIFO Page Replacement

Page 1	→	1	- -
Page 2	→	1	2 -
Page 3	→	1	2 3
Page 4	→	4	2 3
Page 1	→	4	1 3
Page 2	→		

Enter Number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter frame capacity: 3

FIFO Page replacement

Page 1 → 1 – –
Page 2 → 1 2 –
Page 3 → 1 2 3
Page 4 → 4 2 3
Page 1 → 4 1 3
Page 2 → 4 1 2
Page 5 → 5 1 2
Page 1 → 5 1 2
Page 2 → 5 1 2
Page 3 → 5 3 2
Page 4 → 5 3 4
Page 5 → 5 3 4

Total page faults = 9

LRU Page Replacement

Page 1 → 1 – –
Page 2 → 1 2 –
Page 3 → 1 2 3
Page 4 → 4 2 3
Page 1 → 4 1 3
Page 2 → 4 1 2
Page 5 → 5 1 2
Page 1 → 5 1 2
Page 2 → 5 1 2
Page 3 → 3 1 2
Page 4 → 3 4 2
Page 5 → 3 4 5

Total Page Fault = 10

Optimal Page Replacement

Page 1 → 1 _ _
Page 2 → 1 2 _
Page 3 → 1 2 3
Page 4 → 1 2 4
Page 1 → 1 2 4
Page 2 → 1 2 4
Page 5 → 1 2 5
Page 7 → 1 2 5
Page 2 → 1 2 5
Page 3 → 3 2 5
Page 4 → 4 2 5
Page 5 → 4 2 5

Total Page Fault = 7

Son pg